

LLM-VC-Dungeon

Final Capstone Report

Arad Fadaei, Mahboobeh Yasini, Johnpaul Tamburro

Version 1.0, April 10, 2026

Table of Contents

Abstract	1
Introduction	2
The Problem	2
The Vision	2
Scope and Contributions	2
Report Structure	3
Project Evolution	4
Milestone 1: Project Refinement and Technical Strategy	4
Milestone 2: Core Gameplay Loop and Hybrid Architecture	4
Milestone 3: Programmatic Core Migration	5
Milestone 4: Merge Integration and Runtime Hardening	5
System Architecture	7
Overview	7
Technology Stack	7
Component Breakdown	7
Runtime Modes	7
Data Flow for a Single Turn	8
Deployment	8
Implementation	9
Backend Game Engine	9
Game State Model	9
Turn History and Summarization	9
Action Execution	9
Intent Classification	10
Programmatic Parser	10
LLM Tool-Binding Classifier	10
Dungeon Generation	10
Topology	10
LLM Generation Path	11
Programmatic Generation Path	11
Incremental Expansion	11
LLM Integration	11
Frontend Terminal UI	11
Testing and Validation	13
Test Module Breakdown	13
Intent Parser Tests	13
Game Turn Tests	13
Generator Tests	14

API Map Tests	14
What Was Not Tested	14
Results	15
Delivered Functionality	15
Playable State	15
Runtime Mode Behavior	16
Test Results	16
Challenges and Lessons Learned	17
De-scoping Voice Input	17
LLM Reliability on the Critical Path	17
State Consistency	18
Team Coordination	18
Future Work	19
Voice Input and Audio Output	19
Frontend Automated Testing	19
Intent Classification Accuracy Benchmark	19
Expanded Combat and Item Systems	19
Persistent Cloud Save and Multiplayer	19
Incremental Streaming Responses	20
Conclusion	21

Abstract

LLM-VC-Dungeon is a web-based text adventure game that replaces rigid command-parser syntax with natural language input, allowing players to describe their intentions in plain speech and receive dynamically generated narrative responses. The project was developed over four iterative milestones, beginning with a voice-first multimodal concept and converging on a robust, fully containerized full-stack application backed by a deterministic game engine with optional LLM augmentation.

The system is composed of a FastAPI backend that manages game state, intent classification, and dungeon generation, paired with a SvelteKit terminal-style frontend. A three-mode runtime configuration, programmatic, hybrid, and strict-LLM, allows the engine to operate reliably without any LLM provider while preserving richer narrative output when one is available. Multi-provider LLM support targets OpenAI, Google Gemini, and self-hosted Ollama models.

The final deliverable supports a complete game loop: players can explore procedurally generated dungeons, fight enemies, collect items, query lore, save and restore sessions, and view an ASCII map of discovered rooms. A backend test suite of 43 tests across four modules validates intent parsing, turn resolution, generation correctness, and API safety. This report documents the technical design of the final system, the architectural decisions made across each milestone, and the lessons learned from migrating a heavily LLM-dependent prototype into a reliable, deterministic core.

Introduction

The Problem

Traditional text-based adventure games present players with a deceptively simple interface that carries a significant hidden cost: memorization. Classics of the genre, and even modern interactive fiction, rely on rigid verb-noun command parsers that accept only specific syntactic forms. A player who types "examine the chest" when the parser expects "look chest" is met with a failure message. One who tries "use torch on door" when the system wants "light door" is similarly blocked. This cognitive overhead shifts the player's attention away from the narrative and onto the mechanics of communicating with the machine, breaking immersion precisely when it should be strongest.

The effects of this friction are well understood. Players either memorize command vocabularies, consult in-game help repeatedly, or abandon play entirely when their natural phrasing goes unrecognized. The interface becomes an obstacle rather than a transparent medium for storytelling.

The Vision

The goal of this project is to remove that obstacle entirely. LLM-VC-Dungeon is designed so that players need not learn any command syntax. Instead, they type natural language: "I want to search the room carefully" or "attack the goblin with my sword" or "what do I know about this ruin?" The system resolves these inputs into structured game actions, advances the world state deterministically, and optionally returns narrated responses in the voice of a dungeon master.

The original concept went further still: a "Voice First" design where players would speak aloud and receive synthesized audio narration in return, making the interface truly invisible. That vision informed the early architecture and remains a long-term aspiration, but the project ultimately de-scoped live voice input in favor of a stable, text-based natural language pipeline. The reasoning behind that decision is documented fully in the challenges section of this report.

Scope and Contributions

This project is a full-stack web application. The backend is a FastAPI service responsible for all game logic, LLM orchestration, and persistence. The frontend is a SvelteKit terminal-style UI that handles player input and renders game output. The two communicate over a standard HTTP API. Deployment is containerized via Docker Compose.

The primary technical contributions of the project are:

- A deterministic game engine for a text adventure that handles movement, combat, item management, and lore queries without requiring any LLM at runtime.
- A flexible intent classification layer that switches between a rule-based parser and LLM tool-binding depending on runtime configuration.
- A hybrid dungeon generator that uses procedural biome tables as a reliable foundation and LLM-generated content as an optional enrichment layer.

- A multi-provider LLM integration with quota-aware throttling and graceful fallback.
- A 43-test backend validation suite covering the full game turn lifecycle.

Report Structure

The remainder of this report is organized as follows. Section 3 traces the project’s evolution across all four milestones, explaining the key architectural pivots and the reasoning behind them. Section 4 describes the final system architecture. Section 5 details the implementation of each major subsystem. Section 6 presents the testing strategy and validation results. Section 7 describes the final state of the deliverable. Section 8 reflects on the challenges encountered and lessons learned. Section 9 outlines directions for future work, and Section 10 concludes.

Project Evolution

The project was developed over four milestones, each producing a written report. Taken together, they trace a path from an ambitious voice-first concept through two major architectural pivots to a stable, tested, containerized deliverable. This section summarizes each milestone and explains the decisions that shaped the final design.

Milestone 1: Project Refinement and Technical Strategy

The first milestone established the project charter. The core problem, syntax friction in text adventure games, was defined clearly, and the proposed solution was a "Voice First" web application in which players speak naturally and receive synthesized audio narration from an AI dungeon master.

A dual-stack architecture was designed to support both commercial and self-hosted deployment. The commercial stack targeted Groq for speech-to-text and narrative generation, a local Function Gemma model for structured intent logic, and Lemonfox for text-to-speech. The self-hosted stack substituted Faster-Whisper, a local Llama 3 model, and Piper TTS, enabling fully offline operation for research or privacy-sensitive contexts.

A financial model projected variable API cost at approximately \$0.00063 USD per game turn, establishing that the system was commercially viable at scale. A rate limiter and per-session turn limit were identified as necessary safeguards against budget overruns.

The milestone also defined a data strategy for evaluating intent classification: a synthetic dataset of 200 to 300 natural language command / expected function-call pairs, generated by a stronger model such as GPT-4 and used to benchmark prompt engineering accuracy. This evaluation dataset was ultimately not built during the project; the intent classification validation was handled instead through the backend test suite introduced in Milestone 3.

The team was divided into three specializations: frontend, AI engineering, and integration, and a WebSocket architecture was specified for persistent player connections to avoid serverless timeout constraints.

Milestone 2: Core Gameplay Loop and Hybrid Architecture

The second milestone introduced the features that define the game's interface. The most significant was the hybrid command system: a distinction between system-level slash commands, such as `/save`, `/load`, `/theme`, and `/help`, which are handled deterministically by the frontend without any API call, and narrative natural language inputs, which are forwarded to the backend for LLM processing.

The generation strategy was also restructured. Rather than relying on the LLM to construct dungeon content from scratch on every turn, a procedural Python layer was introduced to generate

room topology as a connected graph. The LLM's role was narrowed to adding narrative flavor on top of that deterministic structure. This separation proved to be the conceptual foundation for the more complete migration that followed in Milestone 3.

On the frontend, Svelte 5 rune-based reactive stores were adopted for invisible state tracking. On the backend, Ollama was selected as the primary local inference runtime during development, providing cost-free, private model access with easy provider switching. The game state response contract was expanded to include player and room snapshots alongside the narrative reply.

Voice output via TTS was prototyped but integration complexity and latency constraints foreshadowed its eventual de-scoping.

Milestone 3: Programmatic Core Migration

The third milestone was the most architecturally significant. Previous iterations had validated the LLM-driven gameplay concept, but intent classification and room expansion were still tightly coupled to model responses. The latency of round-tripping every player input through an LLM, combined with the inconsistency of structured JSON output from smaller models, made the system unreliable as a game foundation.

The response was a full migration of core gameplay to deterministic Python. Three key systems were rebuilt without LLM dependency:

A runtime mode flag, `GAME_MODE`, was introduced with three values: `programmatic` for fully deterministic operation, `hybrid` for parser-first with LLM fallback, and `llm` for strict LLM-only classification. A companion flag, `ENABLE_LLM_NARRATION`, decoupled narrative enrichment from gameplay correctness. This allowed the game to run correctly at zero LLM cost while preserving the option for richer output when a provider was available.

A dedicated intent parser replaced the LLM classifier for common actions. The parser handles movement with direction aliases, item and enemy targeting via token-overlap scoring, and conversational input normalization. It returns a structured intent object identical in schema to the LLM classifier's output, enabling transparent interoperability.

Procedural generation was expanded from simple random content to structured biome tables. Each dungeon theme maps to a biome: mine, crypt, arcane, or volcanic; and each biome carries tables of themed descriptors, features, items, and enemies. Room difficulty scales by position in the dungeon, governing item and enemy spawn rates and enemy HP bands.

A backend test suite was introduced covering intent parsing, game turn behavior, and generation correctness.

Milestone 4: Merge Integration and Runtime Hardening

The fourth milestone consolidated everything into a deployable, production-grade baseline. The scope was driven by the integration of a large pull request encompassing 28 changed files, touching backend, frontend, tests, and deployment.

Containerization was completed: `Dockerfile.backend`, `Dockerfile.frontend`, and `docker-compose.yml` define the full deployment stack, with explicit health checks and an optional Ollama Compose profile for local model workflows. Environment configuration was standardized via `.env.example`.

API behavior was hardened in several areas. The save and load paths now enforce directory boundary checks, preventing path traversal attacks by sandboxing all file operations within the `data/` directory. New endpoints were added for game loading (`POST /api/load-game`) and map retrieval (`GET /api/game/map`). The turn response was expanded to include the current room ID, player HP and maximum HP, inventory size, and history size, giving the frontend reliable state context for diagnostics and rendering.

The game logic itself matured: a deterministic `health` action was added, combat now includes enemy retaliation and explicit game-over signaling when player HP reaches zero, visited rooms return distinct messaging, and the `query` action stores extracted facts on entities for persistence across turns.

The test suite grew from its Milestone 3 introduction to 43 total backend tests. The frontend gained `/load-game` and `/map` commands, with the map command rendering an ASCII grid of discovered rooms.

System Architecture

Overview

LLM-VC-Dungeon is a client-server web application. The backend is a Python FastAPI service that owns all game state, intent resolution, dungeon generation, and LLM orchestration. The frontend is a SvelteKit single-page application that presents a terminal-style interface to the player. The two communicate over a standard HTTP REST API. Both services are containerized and launched together via Docker Compose.

Technology Stack

The backend is written in Python and uses FastAPI as the web framework, with Pydantic models providing schema validation throughout. LLM orchestration is handled by LangChain, which abstracts provider differences and provides the tool-binding infrastructure used for structured intent classification. Game state is persisted as JSON files in a sandboxed `data/` directory.

The frontend is written in TypeScript using SvelteKit with Svelte 5 runes for reactive state management. The terminal UI is rendered in the browser and communicates with the backend through a typed API client. No frontend framework or UI component library is used; the terminal appearance is built directly in Svelte components.

LLM providers are pluggable at runtime via environment variables. The system supports OpenAI (GPT-4o-mini), Google Gemini (gemini-1.5-flash for classification, gemini-2.5-pro-preview for generation), and Ollama (functiongemma, qwen3:8b and others) for local self-hosted inference.

Component Breakdown

The backend is organized into four primary modules. The game engine (`game.py`) owns the turn resolution pipeline: it receives a player input, coordinates intent classification, executes the resolved action against the world state, optionally requests narrative enrichment from the LLM, and returns a structured response. The intent module (`intent.py`) implements both the deterministic parser and the LLM tool-binding classifier. The generator (`generator.py`) handles dungeon topology creation and room content population using both procedural tables and LLM generation. The LLM service (`llm.py`) provides a unified interface over all supported providers, including quota detection and throttle recovery.

The frontend is organized around a core `Terminal.svelte` component that handles input dispatch and output rendering. A `commandSystem.ts` module routes slash commands to their handlers deterministically, while natural language inputs are forwarded to the backend API. A typed `client.ts` handles all API communication. Reactive state, including player HP, inventory, and room metadata, is managed with Svelte 5 runes.

Runtime Modes

The system's behavior is governed by two environment flags. `GAME_MODE` accepts three values:

`programmatic` runs the parser-only path with no LLM calls anywhere in the turn pipeline; `hybrid` runs the parser first and falls back to LLM classification only for inputs the parser cannot resolve; `llm` requires a configured LLM provider and fails loudly if one is unavailable. A second flag, `ENABLE_LLM_NARRATION`, independently controls whether narrative enrichment is requested after turn resolution. This means, for example, that a hybrid session can produce plain deterministic output while still routing ambiguous intent to the LLM, or that a fully programmatic session can receive enriched narrative if the narration flag is set and a provider is available.

Data Flow for a Single Turn

A single player turn flows through the system as follows. The player submits text input through the terminal. If the input begins with `/`, the frontend dispatches it to the command handler directly. Otherwise, the input is sent to `POST /api/game/turn` with a session identifier. The backend retrieves the current `GameState` from the JSON save file, runs intent classification according to the configured mode, executes the resolved action to produce a canonical result string, optionally calls the LLM for narrative enrichment, appends the turn to the history log, persists the updated state, and returns a structured response containing the narrative, current room ID, player HP, inventory size, and game-over flag. The frontend updates its reactive state from the response fields and appends the output to the terminal display.

Deployment

Docker Compose defines two services: `backend` and `frontend`. The backend container exposes the FastAPI app via Uvicorn on port 8000. The frontend container serves the SvelteKit build via Node on port 3000. An optional third service profile activates an Ollama container for fully self-contained local inference. Health check endpoints on both services allow the frontend diagnostics screen to report live provider and endpoint status.

Implementation

Backend Game Engine

The game engine is the central coordinator of the backend. Its primary entry point is `process_turn()` in `game.py`, which accepts a player input string and a game state object and returns a completed turn response. The engine's contract is strict: regardless of whether LLM services are available, `process_turn()` must return a valid response with a canonical action result.

Game State Model

All game state is represented by a hierarchy of Pydantic models defined in `models.py`. The root object, `GameState`, holds the current theme string, a `PlayerState`, a dictionary of rooms keyed by room ID, a history list of action/result strings, and a `history_summary` string. `PlayerState` carries the player's current and maximum HP, their inventory as a list of `Item` objects, and their current room ID. Each `Room` holds a description, a map of directional exits to room IDs, lists of `Item` and `Enemy` objects, and an `extra_info` dictionary for persisted facts. Items carry a name, description, and type from the `ItemType` enum. Enemies carry a name, HP and maximum HP, type from the `EnemyType` enum, a description, and their own `extra_info` dictionary.

The `extra_info` dictionaries on rooms, items, and enemies are the foundation of the persistent fact system. When a player asks a factual question about an entity, the engine stores the answer there and returns it on subsequent queries to the same entity, giving the illusion of a consistent world without repeated LLM calls.

Turn History and Summarization

The history list grows with each turn as a string pair of the form `"Action: <input> | Result: <narrative>"`. The engine retains the most recent eight turns in full and rolls older turns into a rolling `history_summary` string, which is truncated to 700 characters. When narrative enrichment is requested, the LLM receives the full recent window plus the summary as context, bounding the context window growth regardless of session length.

Action Execution

After intent classification resolves the player's input to a structured action, the engine executes it against the world state:

- **Move:** validates that the requested exit exists on the current room, logs a distinct "You return to..." message for previously visited rooms, triggers background generation of unvisited neighbors, and updates the player's current room ID.
- **Look:** returns the current room's description along with visible enemies and items.
- **Take:** matches the player's target string against item names using token-overlap scoring, adds the matched item to inventory, and removes it from the room.
- **Attack:** resolves combat between the player and a matched enemy. Enemies with ten or fewer HP are defeated in one hit; stronger enemies take four to twelve points of random damage per

round. Surviving enemies retaliate for two to eight damage. When player HP reaches zero, the `game_over` flag is set in the response.

- **Inventory:** returns the contents of the player's inventory.
- **Health:** returns the player's current and maximum HP.
- **Query:** extracts a subject entity from the input, checks its `extra_info` for a cached answer, and if absent, calls the LLM to generate and store a fact.

Intent Classification

Intent classification is implemented in `intent.py` as two parallel paths that share an output schema.

Programmatic Parser

The parser normalizes the input string by lowercasing and stripping punctuation, then tokenizes it. A set of known verb roots: move, look, take, attack, inventory, health, query, are matched against the normalized tokens. Direction words including cardinal names, abbreviations, and colloquial aliases such as "up" for north and "left" for west are extracted via a lookup table. For actions that require a target, the parser filters stopwords and scores candidate entity names from the current room's items and enemies using token overlap with the remaining input tokens. The highest-scoring match above a minimum threshold is selected as the target. Inputs that match no known verb pattern return an `unknown` action, which triggers the LLM fallback path in hybrid mode.

LLM Tool-Binding Classifier

The LLM path uses LangChain's tool-binding interface to bind a set of `@tool`-decorated functions, one for each supported action, to the language model. The model is prompted with the player input and the current room context and selects the appropriate tool call, returning a structured JSON object that maps directly to the same intent schema as the parser. If the model fails to produce a valid structured response, the classifier retries up to three times before returning an `unknown` fallback.

Dungeon Generation

Dungeon generation is handled by `generator.py` and runs at new game start and incrementally as the player explores.

Topology

The dungeon layout is a ten-room graph generated via BFS on a two-dimensional grid. Starting from a central origin room, the generator expands in random directions, placing rooms and establishing bidirectional exits. The north/south and east/west relationships are always symmetric: a room reachable going north has a south exit back to its source. The resulting graph is connected but not necessarily a simple chain; branching paths and dead ends emerge from the random expansion order.

LLM Generation Path

When an LLM provider is available and configured, the generator issues a single prompt requesting a complete dungeon JSON structure for all ten rooms in one call. The prompt includes the selected theme and a strict output schema. The response is validated against the schema, and malformed or incomplete responses trigger up to three retries before falling back to the programmatic path. The single-call approach is preferable to per-room calls because it allows the LLM to produce thematically coherent content across the whole dungeon rather than rooms that were generated independently.

Programmatic Generation Path

The programmatic path infers a biome from the theme string and populates rooms from biome-specific content tables. Each biome, mine, crypt, arcane, and volcanic, defines tables of room descriptors, environmental features, item names, and enemy names and types, ensuring that content is internally coherent without requiring an LLM. Room difficulty scales with the room's index in the dungeon: early rooms spawn fewer and weaker enemies, while later rooms have higher spawn rates and enemy HP drawn from a higher band. This creates a natural sense of progression without any runtime AI involvement.

Incremental Expansion

When a player moves into a room that has not yet been populated, the engine triggers background generation of the newly entered room and its unvisited neighbors via FastAPI's `BackgroundTasks`. This avoids blocking the turn response on generation while ensuring that the next rooms are ready before the player reaches them.

LLM Integration

The LLM service layer in `llm.py` provides a unified call interface over all supported providers. Provider selection and model identifiers are read from environment variables at startup. The service detects quota exhaustion and rate-limit errors by parsing error messages for patterns such as "retry in 48.3s" or structured `retryDelay` fields. When throttling is detected, the provider is marked unavailable for the extracted delay period, during which all LLM calls short-circuit to the programmatic fallback without attempting network requests. This mechanism prevents cascading failures during API quota events and ensures the game remains playable regardless of provider status.

Frontend Terminal UI

The frontend is built around a terminal metaphor. `Terminal.svelte` renders a scrolling output log and accepts player input through a text field. Submitted input passes through `commandSystem.ts`, which checks for a leading `/` to dispatch slash commands locally and forwards everything else to the backend API client.

System commands cover the full session lifecycle: `/new` starts a new game, `/save` persists the current state, `/load` restores a saved session, `/theme` sets the dungeon theme before generation, `/help` prints available commands, `/map` requests the room graph and renders it as an ASCII grid, and `/api-check`

fetches the health metadata and displays active endpoints and provider status.

Natural language input is sent to `POST /api/game/turn`, and the response is displayed in the terminal with narrative text followed by game state metadata. Svelte 5 runes maintain reactive copies of player HP, inventory size, and current room for display in the terminal header.

Testing and Validation

Testing was introduced in Milestone 3 and expanded substantially in Milestone 4. The final backend test suite contains 43 tests across four modules, run with `pytest` and `pytest-asyncio`. All tests target the programmatic execution paths and mock LLM interactions where needed, making the suite runnable without any active LLM provider.

Test Module Breakdown

Module	Tests	Coverage Focus
<code>test_intent.py</code>	14	Parser correctness across all action types, conversational inflections, LLM tool-binding retry logic, unknown action fallback
<code>test_game_turns.py</code>	18	Move validation and room transitions, item pickup with fuzzy matching, combat with damage and retaliation, multi-turn HP depletion and game-over signaling, history truncation, visited-room return messaging, LLM fallback behavior in hybrid and strict modes, quota/throttle detection
<code>test_generator_programmatic.py</code>	6	Theme selection in deterministic mode, biome-weighted room content, difficulty scaling of enemy HP across room indices, full ten-room dungeon generation roundtrip, strict-LLM error handling when provider is unavailable
<code>test_api_map.py</code>	5	Room graph serialization for the map endpoint, current room tracking in the response, path traversal prevention in save/load paths, history context forwarding to LLM calls

Intent Parser Tests

The fourteen intent tests exercise both breadth and robustness of the parser. Coverage includes every directly supported action: movement with cardinal names, directional abbreviations, and colloquial aliases; item take commands with varying phrasing; attack commands with explicit and implicit targets; inventory and health queries. Additional tests confirm that conversational inflections, phrasing such as "i grab the key" or "i'm attacking," are correctly normalized to their canonical action types. The LLM tool-binding path is tested through mocked provider calls that simulate retry failures, confirming that the classifier returns an `unknown` result after three failed attempts rather than raising an exception.

Game Turn Tests

The eighteen game turn tests validate the full turn pipeline end to end against a synthetic game

state. Tests confirm that movement into a valid exit updates the player's room, that movement into a non-existent direction returns an appropriate error message, and that revisiting a room produces different output than the first visit. Item pickup tests confirm token-overlap matching selects the correct item when the player's wording only partially matches the item name, and that items are removed from the room and added to inventory. Combat tests verify damage calculation, enemy retaliation, and HP mutation across multiple turns, culminating in a game-over flag when HP reaches zero. History truncation behavior is validated by forcing a session past the eight-turn recency window and confirming the summary is populated and recent turns are preserved. Two tests specifically exercise mode-dependent behavior: one confirms that `hybrid` mode falls back to LLM classification for an unknown input, and one confirms that `llm` strict mode raises a hard error when no provider is configured.

Generator Tests

The six generator tests focus on programmatic correctness. A theme generation test confirms that deterministic mode selects a preset theme rather than calling the LLM. Biome content tests confirm that generated rooms contain items and enemies consistent with the chosen biome's tables. The difficulty scaling test compares enemy HP in early rooms against enemy HP in late rooms, verifying that the progression band is applied correctly. A full dungeon roundtrip test generates all ten rooms and asserts structural completeness: ten rooms present, all exits bidirectional, all rooms containing at least one item or enemy. A final test confirms that generating a dungeon with `GAME_MODE=llm` and no provider configured raises a descriptive error rather than silently producing empty content.

API Map Tests

The five API map tests validate the map endpoint contract and the security of the file system access layer. Tests confirm that the `/api/game/map` response includes the correct number of rooms and correctly identifies the player's current room. The most important security test constructs a load request with a path traversal payload, a filename containing `../` segments, and asserts that the backend rejects it rather than reading an arbitrary file, confirming the `os.path.commonpath()` boundary check is effective.

What Was Not Tested

Frontend behavior is not covered by the test suite. End-to-end interaction between the Svelte terminal and the backend API was validated manually during development but no automated integration tests exist. The intent classification evaluation dataset described in Milestone 1, a formal benchmark of natural language input accuracy, was not constructed. These represent the primary gaps in validation coverage and are noted as areas for future work.

Results

Delivered Functionality

The final deliverable is a fully playable, containerized text adventure game with a complete game loop and a stable API contract. The following table summarizes feature status against the original and evolved project scope.

Feature	Status	Notes
Natural language input	Delivered	Text-based; voice input de-scoped
Intent classification	Delivered	Programmatic parser + optional LLM tool-binding
Dungeon generation	Delivered	Procedural biome tables + optional LLM single-call path
Movement and exploration	Delivered	BFS 10-room graph with bidirectional exits
Combat system	Delivered	Deterministic damage/retaliation, game-over signaling
Item pickup and inventory	Delivered	Token-overlap fuzzy matching
Lore query with fact persistence	Delivered	<code>extra_info</code> stored on rooms, items, enemies
Save and load	Delivered	JSON persistence with path traversal protection
ASCII map display	Delivered	<code>/map</code> command renders discovered rooms as grid
LLM narrative enrichment	Delivered	Optional; multi-provider with quota/throttle fallback
Multi-provider LLM support	Delivered	OpenAI, Gemini, Ollama
Containerized deployment	Delivered	Docker Compose with optional Ollama profile
Backend test suite	Delivered	43 tests across 4 modules
Voice input (STT)	De-scoped	See Section 8
TTS audio output	De-scoped	See Section 8
Frontend automated tests	Not delivered	Manual validation only
Intent classification benchmark dataset	Not delivered	Planned in M1, superseded by unit tests

Playable State

A new game session begins with the player selecting a theme, or accepting a generated one, after which the dungeon is populated and the player is placed in the starting room. The terminal displays

the room description, visible items, and available exits. The player can type freely: movement commands in any phrasing, combat commands referencing enemies by partial name, item pickup requests, inventory and health queries, and lore questions are all resolved without syntax guidance.

As the player moves through the dungeon, neighboring rooms are populated in the background. Enemies retaliate when attacked and track HP across turns. Items collected in one room persist in inventory across sessions if the player saves. The ASCII map builds incrementally as rooms are visited. When player HP reaches zero, the session ends with a game-over message.

Runtime Mode Behavior

In **programmatic** mode, all of the above works without any network request beyond the initial API call to the backend. Turn latency in this mode is bounded by local Python execution, typically under 50 milliseconds on a standard development machine. In **hybrid** mode, latency is the same for recognized inputs and adds one LLM round-trip only for inputs the parser cannot classify. In **llm** mode, every turn passes through the LLM, making narration the primary output but introducing provider latency on the critical path.

Test Results

All 43 backend tests pass. The test suite runs in under ten seconds on a standard development machine. No known test failures exist at the time of submission.

Challenges and Lessons Learned

De-scoping Voice Input

The most visible departure from the original project vision was the removal of speech-to-text input and text-to-speech output. The Milestone 1 design called for a fully voice-first interface: the player would speak to the browser, and the game would respond with synthesized audio narration. By Milestone 2, voice output via TTS had been prototyped but not integrated. By Milestone 3, voice input as a primary interface had been quietly set aside in favor of text.

Several factors drove this decision. The Web Audio API imposes browser-specific restrictions on microphone access that vary across Chrome, Firefox, and Safari. Handling the full permissions lifecycle, stream management, and error recovery added substantial frontend complexity that was orthogonal to the game's core value. Speech-to-text transcription introduced a second layer of latency before intent classification could even begin, making the total round-trip time for a single player turn potentially in the three-to-five-second range on a consumer connection with a remote STT provider. And unlike a wrong classification, a transcription error is opaque to the player: they do not see the misheard text, so they cannot distinguish a game logic failure from a transcription failure. Text input eliminates that uncertainty entirely.

The de-scoping was a pragmatic scope decision, not an abandonment of the vision. The natural language classification layer, which was always the harder problem, was built and validated. Voice input is a frontend integration layer on top of that classification capability and remains a viable extension.

LLM Reliability on the Critical Path

The most significant technical challenge of the implementation phase was that LLMs, when placed on the critical path for every game turn, are unreliable in ways that are difficult to handle gracefully. The problems manifested in three forms.

The first was structured output consistency. Smaller models, including local Ollama models in the 7B to 8B parameter range, frequently produced intent classifications that failed schema validation: missing fields, incorrect types, or action names that did not match the expected enum values. Retry logic with exponential backoff helped, but a multi-retry pipeline on every turn compounded latency. The programmatic parser solved this completely for the common cases, reducing the LLM classification path to a fallback used only for genuinely ambiguous inputs.

The second was latency. Even with a local model via Ollama, LLM inference for a single classification call took 800 to 1500 milliseconds on development hardware during active generation. For a game whose state changes on every input, that round-trip made the interface feel unresponsive. Moving classification to deterministic Python dropped turn latency to under 50 milliseconds for recognized inputs.

The third was quota exhaustion. Cloud LLM providers impose rate limits and daily quotas that, during active development with multiple simultaneous sessions, were hit without warning. An unhandled quota error would crash the game turn entirely. The quota detection and cooldown

mechanism in `llm.py` was built in direct response to several interrupted test sessions, and the design decision to make `GAME_MODE=programmatic` an option was a direct consequence of this experience.

The broader lesson is that production LLM integrations require a fallback at every point where the model sits in a required code path. The hybrid architecture developed here, deterministic logic first, LLM enrichment only where it adds value and where failure is recoverable, is the pattern that should have been the starting design rather than an evolution reached through trial and error.

State Consistency

Early iterations placed too much responsibility on the LLM for maintaining game state. Responses included inventory references, room state descriptions, and entity facts generated at inference time rather than drawn from a persistent model. These LLM-generated state references frequently diverged from the actual game state tracked in the backend, producing situations where the narrative told the player they had picked up an item while the inventory list showed it was still in the room. The resolution was straightforward once the problem was understood: the deterministic engine owns all state mutations, and the LLM is given only the current verified state as context for enrichment. It may describe what it is told, but it cannot invent state that the engine does not confirm.

Team Coordination

The project was developed across three roles with distinct technical domains. The separation of frontend, AI engineering, and backend integration responsibilities worked well for parallel development but created integration friction at the API boundary. Early API contracts were implicit and changed without coordinated updates, causing frontend components to break silently when backend response shapes evolved. The Milestone 4 work of formalizing the turn response schema and adding explicit field documentation resolved the majority of these coordination issues. The lesson is that API contract documentation is not a finishing step; it is a precondition for effective parallel work.

Future Work

Voice Input and Audio Output

Restoring the original voice-first vision remains the most significant open direction. The natural language classification layer is complete; adding voice input is a matter of integrating a browser speech-to-text pipeline that feeds text into the existing turn API. The architecture is already designed for it: the frontend command system routes text strings to the backend regardless of whether they originated from keyboard input or transcription. A Faster-Whisper self-hosted STT integration or a browser-native Web Speech API binding would both be compatible with the current backend without modification. TTS output would similarly attach as a post-processing layer on the narrative response string, calling a provider such as Piper or a cloud TTS API before rendering the terminal line. Completing this integration would realize the Milestone 1 vision.

Frontend Automated Testing

The test suite covers the backend thoroughly but has no frontend equivalent. A Playwright end-to-end suite would allow automated validation of the full turn flow from browser input to terminal output, including command dispatch, API integration, and reactive state updates. Given that frontend bugs were discovered manually during integration testing, this gap represents genuine risk for continued development.

Intent Classification Accuracy Benchmark

The Milestone 1 data strategy called for a synthetic dataset of 200 to 300 natural language command to expected function-call pairs. This dataset was never built. Constructing it using a strong model to generate diverse phrasings, and running the programmatic parser against it, would give a concrete measurement of parser coverage and identify categories of phrasing the parser fails on. The result would drive targeted improvements to the token-overlap scoring and normalization logic.

Expanded Combat and Item Systems

The current combat system is intentionally minimal: a single damage range applied uniformly across all enemies, with a simple retaliation roll. A richer combat system could include weapon type modifiers from the player's equipped item, enemy-specific resistances and abilities drawn from the `EnemyType` model, status effects such as poisoning or stunting, and multi-turn spell casting. Item types beyond `WEAPON` and `POTION`; `ARMOR`, `KEY`, and `TREASURE` are already defined in the `ItemType` enum but are not yet mechanically active.

Persistent Cloud Save and Multiplayer

Sessions currently persist as local JSON files on the backend host. A database-backed persistence layer would allow sessions to survive container restarts, support multiple concurrent users, and enable cloud-hosted deployment where the backend is not stateful. The `GameState` Pydantic model serializes cleanly to JSON already, making a migration to a document store such as MongoDB

straightforward.

Incremental Streaming Responses

Long narrative responses currently block until the LLM finishes generation before the backend responds. Streaming the LLM output token by token to the frontend via server-sent events or a streaming HTTP response would make the game feel more responsive and would replicate the character-by-character reveal common in AI chat interfaces, which suits the terminal aesthetic well.

Conclusion

LLM-VC-Dungeon set out to solve a specific problem: the syntax friction that makes traditional text adventure interfaces feel like obstacles rather than transparent windows into a story. The answer the project arrived at, natural language input processed by a combination of deterministic rules and optional LLM enrichment, is both more reliable and more maintainable than the original fully LLM-driven design.

What was delivered is a functioning full-stack web application: a text adventure game that accepts free-form natural language, resolves player intent without requiring command memorization, generates procedurally coherent dungeons with or without an LLM provider, and maintains a consistent world state across sessions through a clean persistence model. The project is containerized, documented, and validated by a 43-test backend suite.

The journey to get there involved meaningful architectural pivots. Voice input was de-scoped when the implementation cost outweighed the benefit given the project's timeline. The LLM was moved off the critical game turn path when its unreliability made it unsuitable as a foundation for deterministic game logic. These were not failures of ambition; they were the correct engineering decisions given the constraints, and the reasoning behind each is captured in the milestone record.

The three-mode runtime design: programmatic, hybrid, and strict-LLM, is the result that is most generalizable beyond this project. It provides a practical pattern for building LLM-augmented systems that are reliable by default: the deterministic core is always correct, the LLM layer adds quality where it can, and the system degrades gracefully when the model is unavailable. This pattern applies to any application where LLM output must be layered on top of logic that cannot tolerate hallucination or latency spikes.

The original voice-first vision is incomplete but reachable. The hardest part, teaching the system to understand natural language and translate it into structured game actions, is done. Completing the voice layer is an integration milestone, not a research problem.